

Chapter 9. Domain Models

“It’s all very well in practice, but it will never work in theory.”

Objectives

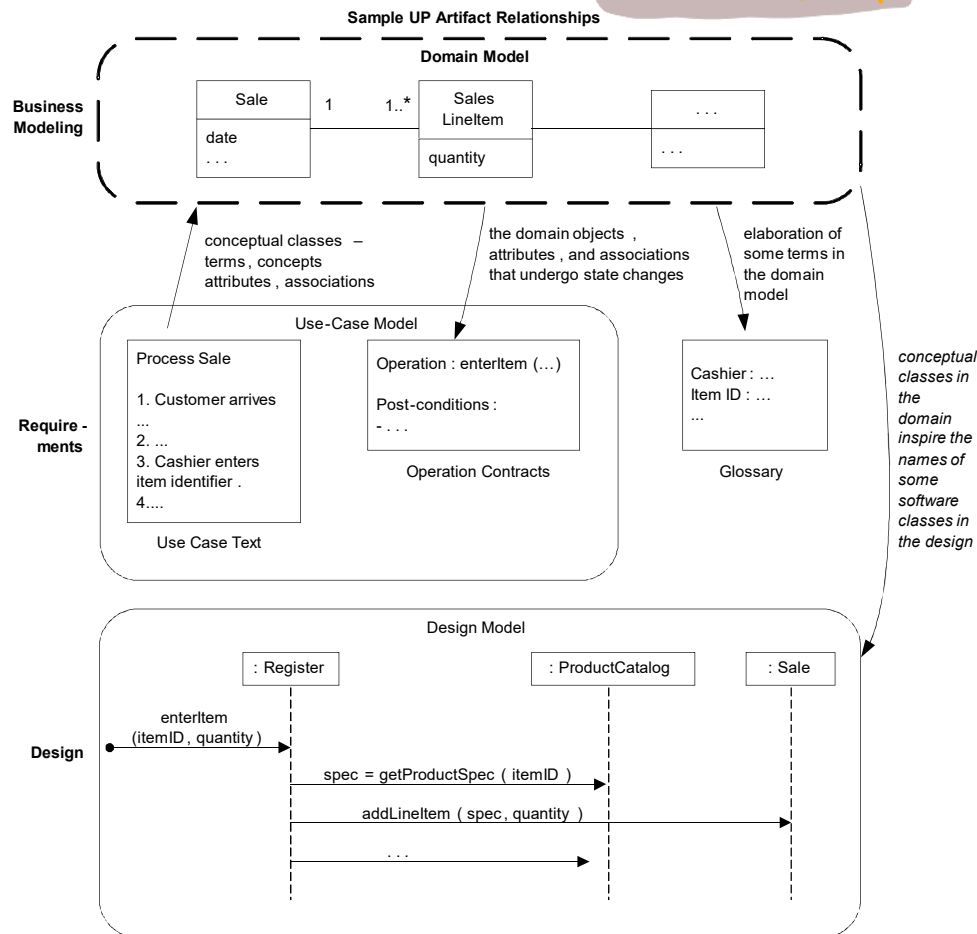
- Identify conceptual classes related to the current iteration.
- Create an initial domain model
- Model appropriate attributes and associations.

Domain Model

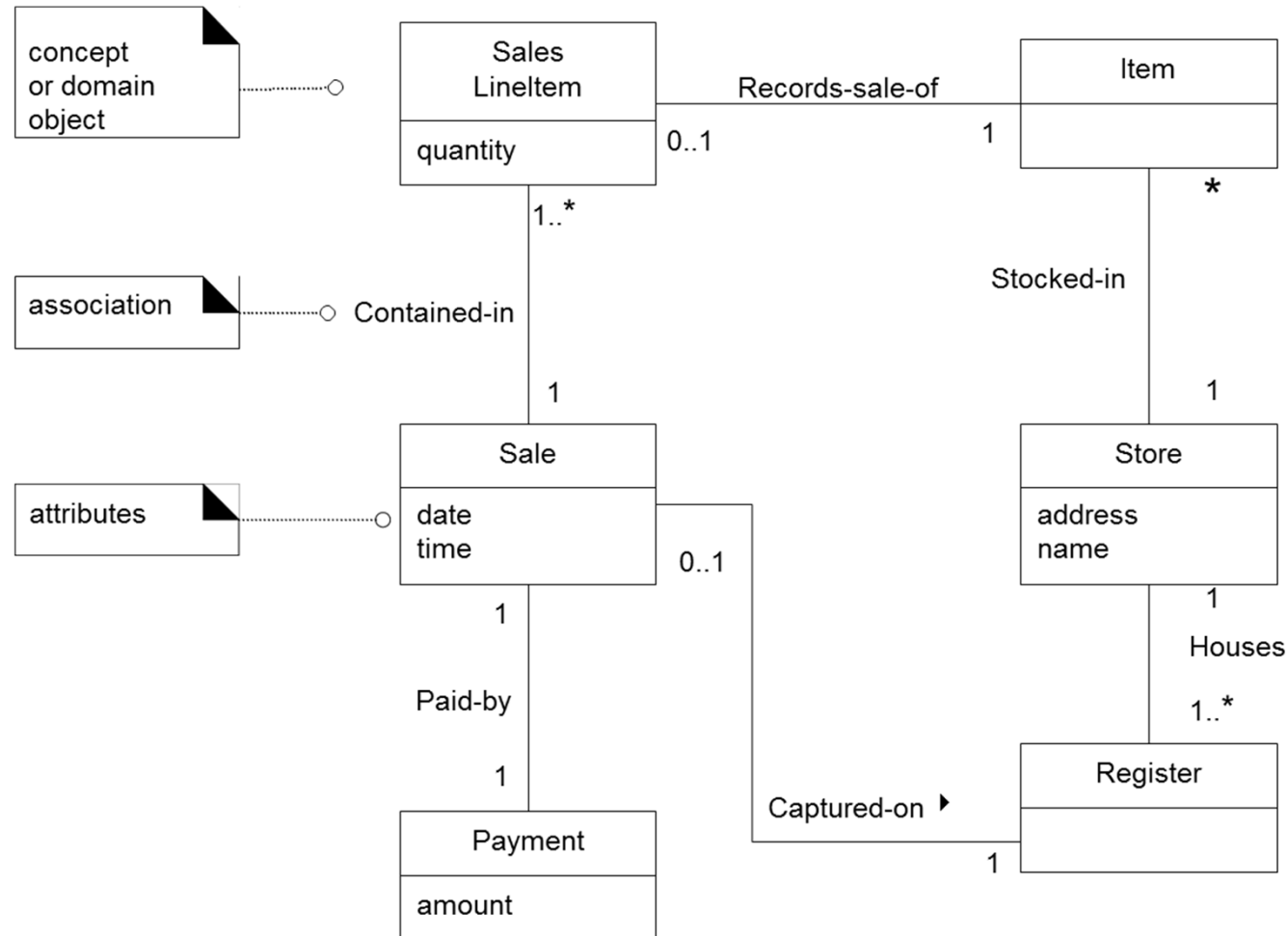
Use = Case를 통해

도메인에서 어떤 object (객체)가 있는지 모델링

- Domain model: illustrates noteworthy concepts in a domain.
 - source of inspiration for designing software objects and classes.
 - The most important classic model in OO analysis!



Partial Domain Model for NextGen POS



What is a Domain Model?

→ 실세계 도메인의 개념적 클래스를 시작하.

- A **domain model** is a **visual** representation of conceptual classes or real-situation objects in a domain.

- Focusing on explaining 'things' and products important to a business domain.
- Also called **conceptual models**, **domain object models**, **analysis object models**

does not contain software objects!

→ 디자인 단계.

- related but, not identical to "domain layer" in software architecture

- "lower representation gap" (later discussed)

- illustrated with a set of **class diagram without operations**

- domain objects (or conceptual classes)
- attributes
- associations

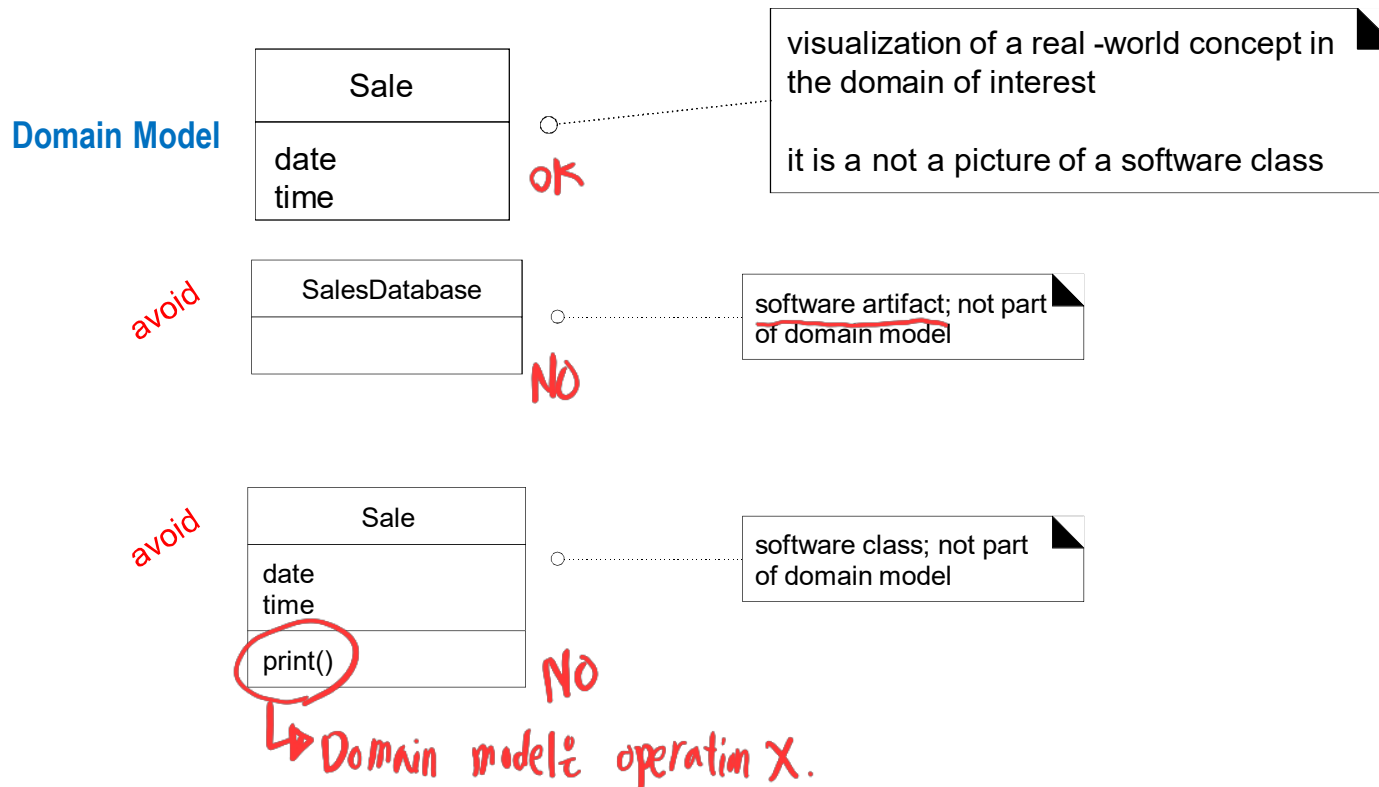
→ 연산 X 인 클래스
다이어그램.

왜냐하면
도메인
관련만 나타.

Domain Model: Picture of Business Objects

- UP domain model

- a visualization of things in a real-situation domain of interest,
- not of ~~software~~ objects such as:
 - Software classes (such C# or Java classes)
 - Elements representing artifacts related to the implementation of the system (e.g., a database or a window) – unless the domain being modeled is of software concepts (e.g., a Graphical User Interface library)
 - Responsibilities or methods



→ Domain Model의 목적 2가지.

Why Create a Domain Model?

- Two reasons to create a domain model:
 1. Understanding the domain during early elaboration iterations
 2. Inspiring the software classes of the **domain layer** in the design model.
 - prevent software from being far away from the reality of the domain
 - lower representation gap (concepts in the domain ~ concepts in the domain layer)
 - Use software class names in the domain layer inspired from names in the domain model, with objects having domain-familiar information and responsibilities. (a very key idea in OO)

1. 초기 Elaboration 단계에서 도메인의 이해를 위해서

2. Design model의 domain layer 클래스에

영감을 주기 위해서.

→ Representation Gap을 줄여줌.

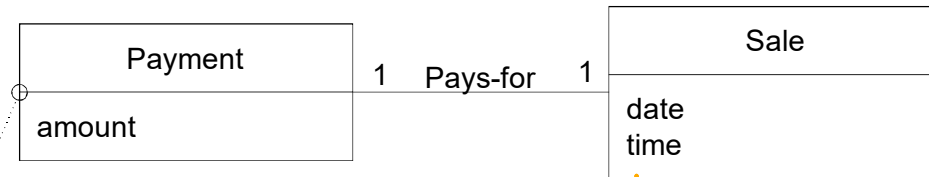
Lower Representation Gap

A Payment in the Domain Model is a concept, but a Payment in the Design Model is a software class. They are not the same thing, but the former *inspired* the naming and definition of the latter.

This **reduces the representational gap**.

This is one of the big ideas in object technology.

UP Domain Model
Stakeholder's view of the noteworthy concepts in the domain.

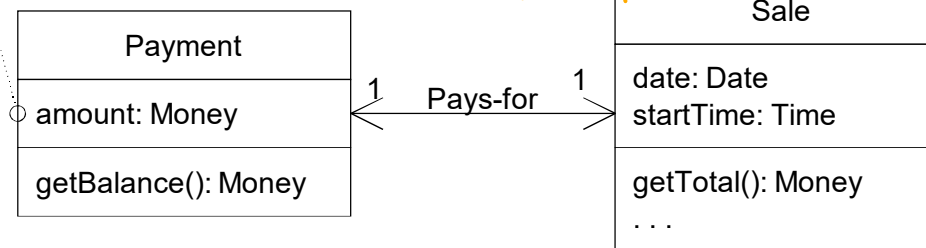


inspires
objects
and
names in

domain에서의 작업이

design 단계에도 영감을 주,

두 단계의 gap을 줄여줌



UP Design Model

The object-oriented developer has taken inspiration from the real world domain in creating software classes.

Therefore, the representational gap between how stakeholders conceive the domain, and its representation in software, has been lowered.

→ 도메인 모델 작성 3단계

Steps to Create a Domain Model

1. Find **conceptual classes**
2. Draw the identified conceptual classes in a **UML class diagram**
3. Add **associations** and **attributes** to conceptual classes

Note: Bounded by the current iteration requirements

1. 개념 클래스 찾기

2. UML 클래스 다이어그램 그리기

3. attribute 와 association 추가

How to find conceptual classes

1. Reusing existing models

- This is the first, best, and usually easiest approach
- There are **published, well-crafted domain models and data models** for many common domains e.g. inventory, finance, health

2. Using a category list

3. Identification of noun phrases

0-1 / 1-3

Method 2 : Use a Category List

Category	Examples
Business transactions	Sale, Payment, Reservation
Transaction line items	SalesLineItem
Product or service related to a transaction or transaction line item	Item, Flight, Seat, Meal
Where is the transaction recorded?	Register, Ledger, FlightManifest
Roles of people or organizations related to the transaction; actors in the use case	Cashier, Customer, Store, MonopolyPlayer, Passenger, Airline
Place of transaction; place of service	Store, Airport, Plane, Seat
Noteworthy events, often with a time or place we need to remember	Sale, Payment, MonopolyGame, Flight
Physical objects	Item, Register Board, Piece, Die, Airplane
Descriptions of things	ProductDescription, FlightDescription

Note: refer to the textbook for more examples

Method 3 : Identification of Noun Phrases 명사구

- A linguistic analysis :
 - Identify the nouns and noun phrases in a textual description of the domain. Consider them as candidate conceptual classes and attributes.
 - Mechanical noun-to-class mapping isn't possible, since natural language is ambiguous.

- Example:

Process Sale use case

domain: | 단어들을 체크해 가면서
생각.

Main Success Scenario:

1. Customer arrives at a POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total. Price calculated from a set of price rules.

Drawing Conceptual Classes in a UML Class Diagram

- After discovering (identifying) the conceptual classes, we can start creating our domain model.
- Example: NextGen POS domain



Initial POS domain model

Report Objects

- *Receipt* is a noteworthy term in the POS domain
- But it's only a report of a sale and payment → duplicate information.
- Decision: current domain model does not include report objects
- Detailed dicussion
 - “Exclude it”
 - In general, showing a report of other information in a domain model is not useful, since all its information is derived or duplicated from other sources.
 - “Include it”
 - It has a special role in terms of the business rules: It usually confers the right to the bearer of the (paper) receipt to return bought items.
 - Since item returns are not being considered in this iteration, *Receipt* will be excluded.
 - During the iteration that tackles the *Handle Returns* use case, we would be justified to include it.

A Common Mistake with Attributes vs. Classes

- If we do not think of X as a number or text in the real world, X is probably a conceptual class, not an attribute.

Sale
store

or... ?

Sale

Store
phoneNumber

- In the real world, a store is not considered a number or text.
- It suggests a legal entity, an organization, and something that occupies space.
- Therefore, *Store* should be a conceptual class.

Flight
destination

or... ?

Flight

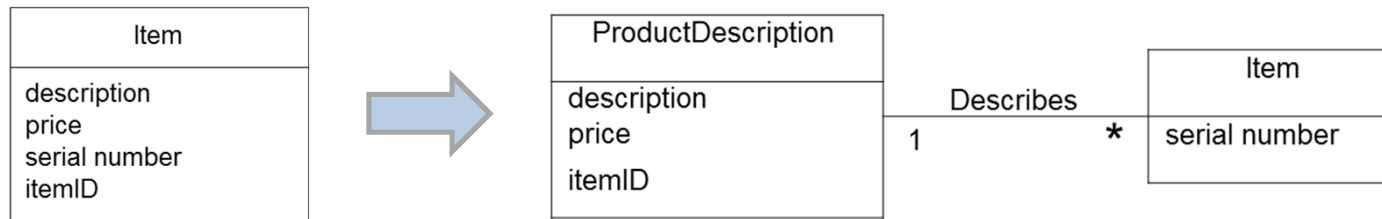
Airport
name

- In the real world, a destination airport is not considered a number or text.
- It is a massive thing that occupies space.
- Therefore, *Airport* should be a concept.

X가 숫자나 문자가 아닌 개념 클래스인 것을 ↑

When to Model with 'Description' Classes?

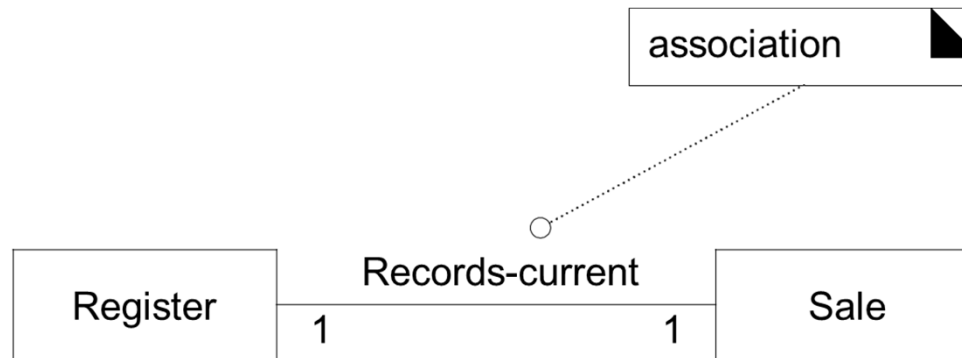
- A **Description class**: contains information that describes something else.
 - Example: a *ProductDescription* that records the price, picture, and text description of an *Item*.



- Add a description class when:
 - There needs to be a description about an item or service, independent of the current existence of any examples of those items or services.
 - Deleting instances of things they describe (for example, *Item*) results in a loss of information that needs to be maintained, but was incorrectly associated with the deleted thing.
 - It reduces redundant or duplicated information.

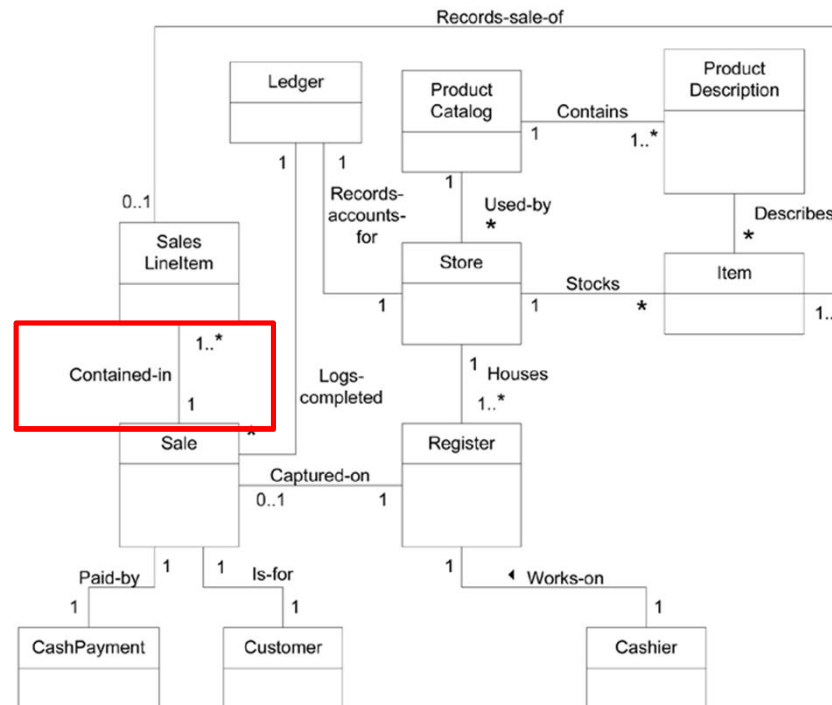
Associations

- **Association** : some meaningful and interesting connection.
 - In the UML, it is defined as “the semantic relationship between two or more classifiers that involve connections among their instances.”



When to Show an Association?

- Consider including the following associations in a domain model:
 - Associations for which knowledge of the relationship needs to be preserved for some duration (“need-to-remember” associations)
 - Associations derived from the Common Associations List (next page)
- Example:
 - We need to remember what *SalesLineItem* instances are associated with a *Sale* instance.
 - Otherwise, it would not be possible to reconstruct a sale, print a receipt, or calculate a sale total.



Common Association List

Category	Examples
A is a transaction related to another transaction B	CashPayment – Sale, Cancellation – Reservation
A is a line item of a transaction B	SalesLineItem – Sale
A is a product or service for a transaction (or line item) B	Item - SalesLineItem (or Sale), Flight – Reservation
A is a role related to a transaction B	Customer – Payment, Passenger – Ticket
A is a physical or logical part of B	Drawer - Register, Square – Board, Seat – Airplane
A is physically or logically contained in/on B	Register- Store, Item- Shelf, Square- Board, Passenger – Airplane
A is a description for B	ProductDescription – Item, FlightDescription – Flight
A is known/logged/recorded/reported/captured in B	Sale – Register, Piece – Square, Reservation – FlightManifest
A is a member of B	Cashier – Store, Player – MonopolyGame, Pilot – Airline
A is an organizational subunit of B	Department – Store, Maintenance – Airline
A uses or manages or owns B	Cashier – Register, Player – Piece, Pilot – Airplane
A is next to B	SalesLineItem – SalesLineItem, Square – Square, City – City

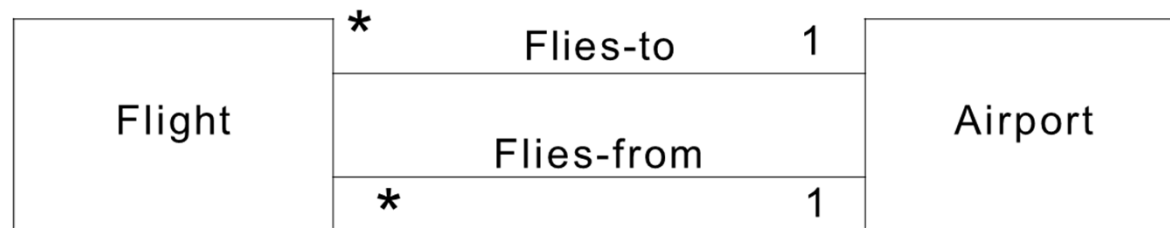
Will the Associations Be Implemented in Software?

→ 그럴 수도 있고 아닐 수도 있다.

- An association in the domain model is meaningful in a purely conceptual perspective in the real domain.
 - Many such relationships will be implemented in software as paths of navigation and visibility
 - But, the domain model is not a data model
 - associations are added to highlight our rough understanding of noteworthy relationships, not to document object or data structures.

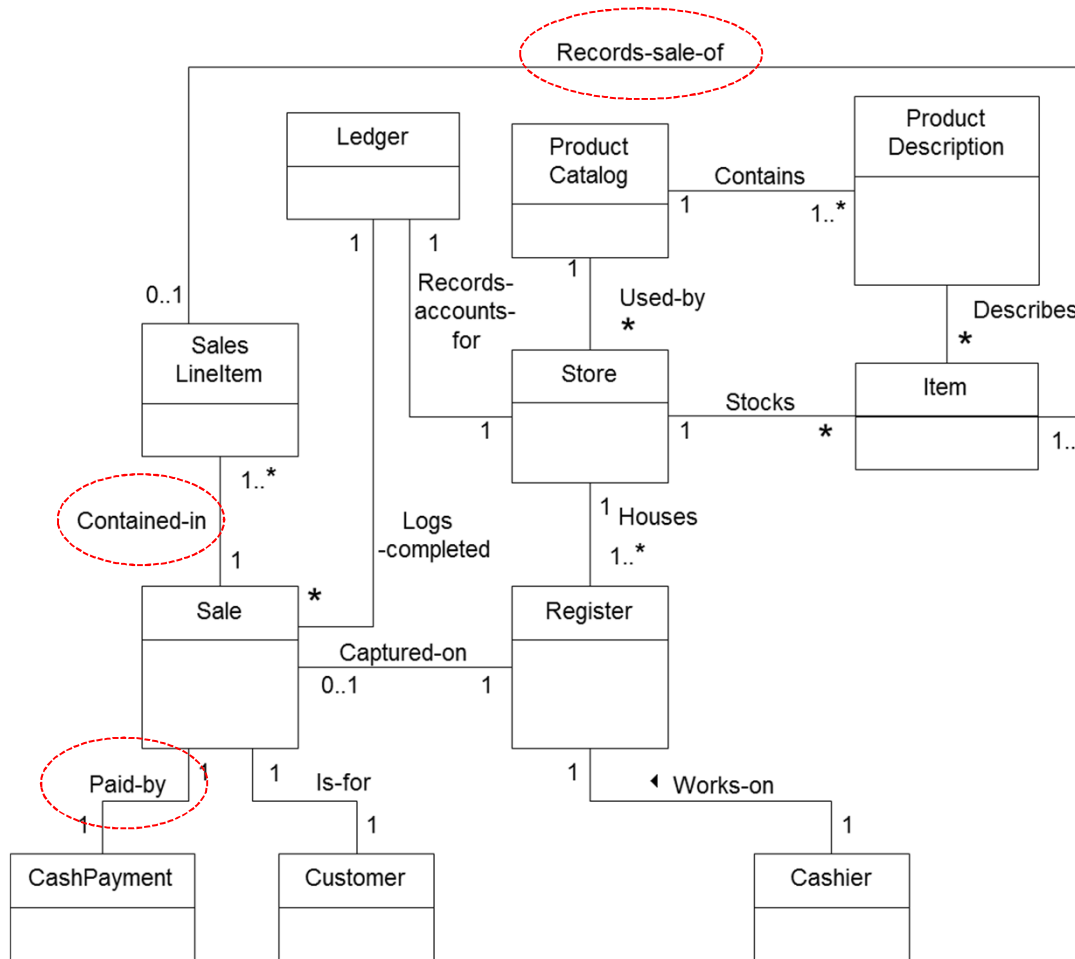
Multiple Associations between Two Classes

- Two classes may have multiple associations between them.



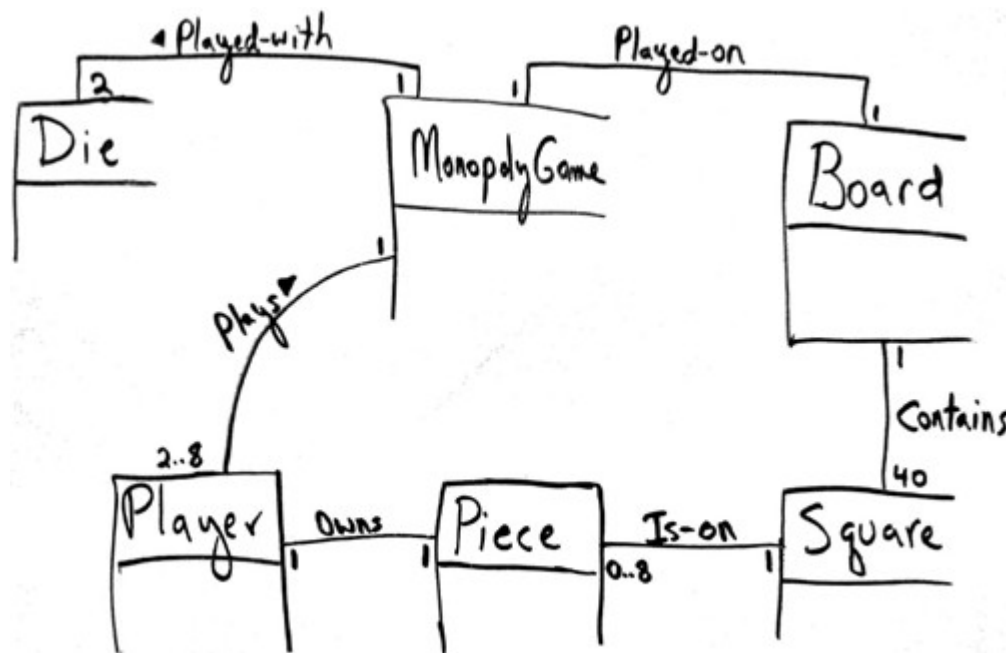
→ multiple association 7/5,

Associations Example: NextGen POS



Associations Example: Monopoly

- **A is contained in or on B** : *Board Contains Square*.
- **A owns B** : *Players Owns Piece*.
- **A is known in/on B** : *Piece Is-on Square*.
- **A is member of B** : *Player Member-of (or Plays) MonopolyGame*.



Attributes

- An **attribute** is a logical data value of an object.
 - It is useful to identify those attributes of conceptual classes that are needed to satisfy the information requirements of the current scenarios under development.
- Attributes are information that we need to remember.
 - *Sale* needs a *dateTime* attribute.
 - *Store* needs a *name* and *address*.
 - *Cashier* needs an *ID*.
- In UML class diagrams, attributes are shown in the second compartment of the class box.

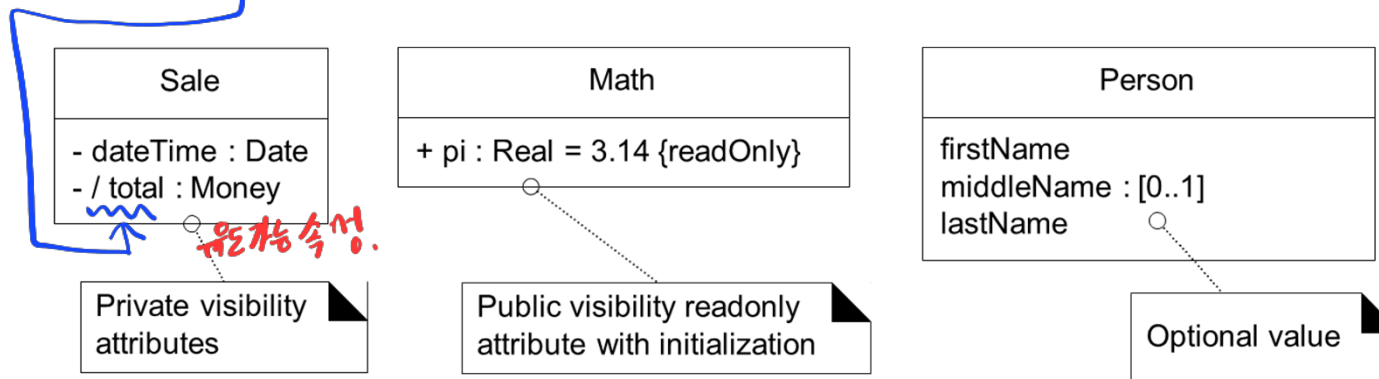
Attribute Notations in UML

- The full syntax for an attribute in the UML is:

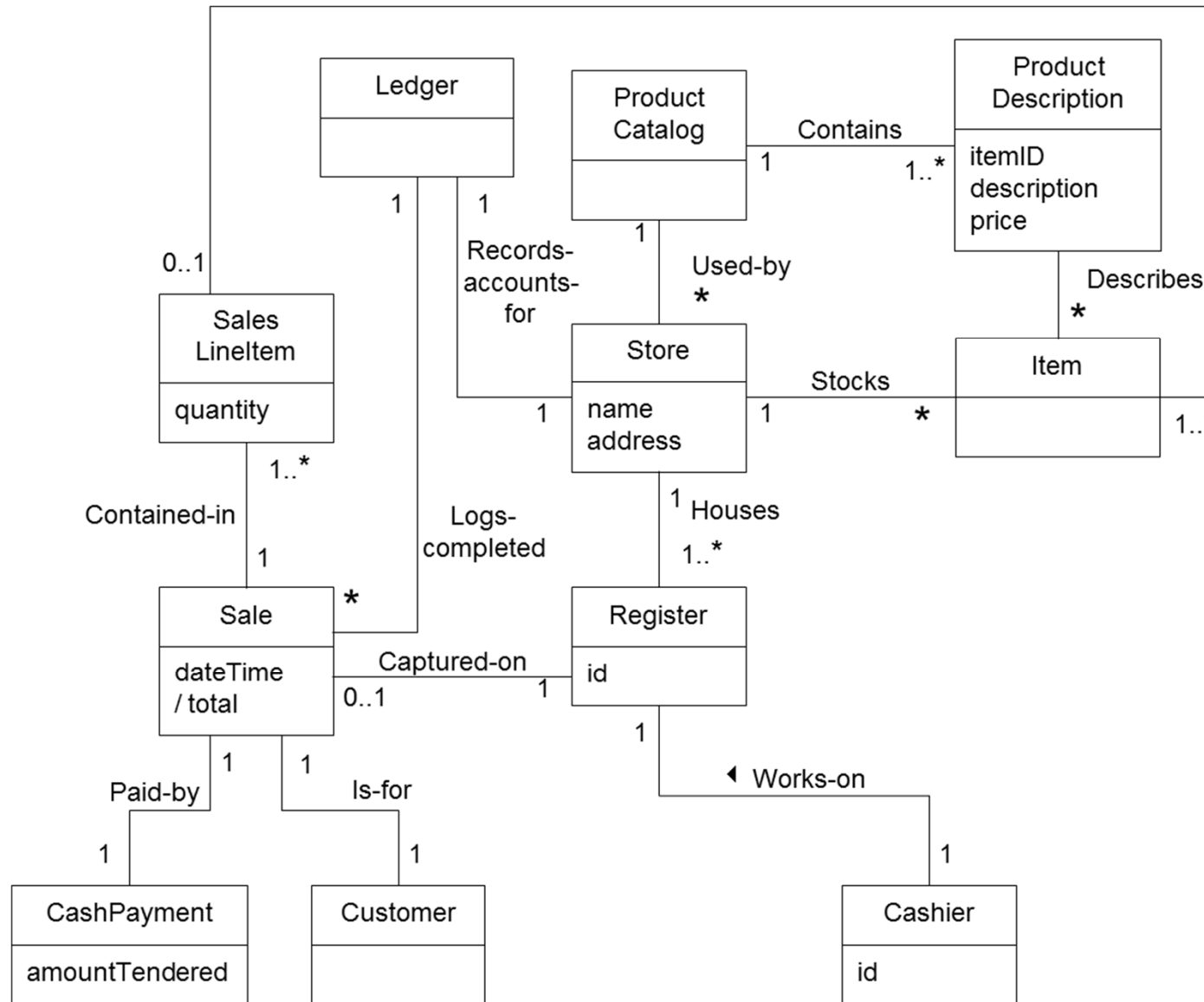
visibility name : type multiplicity = default {property-string}

– Where:

- Visibility: + (public), - (private), # (protected) or ~ (package)
- Name: the name of the variable
- Type: a data type such as Integer, Date, String etc.
- Multiplicity: the multiplicity of the variable (e.g. [0..20])
- Default: the default value for the variable
- Property-String: a property of the variable such as {readOnly} or {frozen}
- If an attribute is derivable, use / **symbol** before the attribute name.



Domain Model for NextGen POS (early version)



UP Domain Model is usually both started and completed in the elaboration

Discipline	Artifact	Incep.	Elab.	Const.	Trans.
	Iteration →	I1	E1..En	C1..Cn	T1..T2
Business Modeling	Domain Model		s		
Requirements	Use-Case Model (SSDs)	s	r		
	Vision	s	r		
	Supplementary Specification	s	r		
	Glossary	s	r		
Design	Design Model		s	r	
	SW Architecture Document		s		
	Data Model		s	r	